C++ 中不同数组返回方式的对比与示例

📌 C++ 函数返回“数组”的正确写法（核心原则：C++ 不允许直接返回数组类型，需用间接方式实现）

1. 最推荐：返回 `std::array`（固定大小数组）

`std::array` 是 C++11 引入的模板类，包装了固定大小的数组，支持值语义，安全且高效，**推荐优先使用**。

Code block

```
#include <array>

// 返回一个包含5个int的数组
std::array<int, 5> get_fixed_array() {
    std::array<int, 5> arr = {1, 2, 3, 4, 5};
    return arr; // 自动拷贝/移动，无内存泄漏风险
}

// 使用示例
int main() {
    auto arr = get_fixed_array();
    for (int x : arr) { /* 遍历 */ }
    return 0;
}
```

✅ 优点：

- 编译期确定大小，性能接近原生数组
- 自动管理内存，无需手动释放
- 支持范围for、迭代器等现代C++特性

⚠ 限制：数组大小必须在编译期指定

2. 最灵活：返回 `std::vector`（动态大小数组）

`std::vector` 是动态数组，大小可在运行时调整，是返回可变长度数组的**首选方案**。

```cpp
#include <vector>

// 返回一个动态大小的数组
std::vector<int> get_dynamic_array(int size) {
    std::vector<int> vec(size);
    for (int i = 0; i < size; ++i) {
        vec[i] = i + 1;
    }
    return vec; // 自动移动，效率极高
}

// 使用示例
int main() {
    auto vec = get_dynamic_array(10);
    vec.push_back(11); // 可动态扩容
    return 0;
}
```

✅ 优点：

- 运行期可调整大小，使用灵活
- 自动管理内存，完全避免内存泄漏
- 标准库广泛支持，生态完善

⚠ 限制：相比 `std::array` 有极轻微的性能开销（可忽略）

3. 基础方案：返回指针（需注意生命周期）

C++ 函数可以返回指向数组的指针，但**绝对不能返回局部数组的指针**（局部数组在函数结束后销毁，指针会变成野指针）。

方式A：返回静态数组指针（生命周期到程序结束）

Code block

```cpp
int* get_static_array() {
    static int arr[5] = {1, 2, 3, 4, 5}; // static 延长生命周期
    return arr;
}
```

方式B：返回动态分配数组指针（需手动释放）

Code block

```cpp
int* get_dynamic_ptr() {
```

```
    int* arr = new int[5]; // 动态分配
    for (int i = 0; i < 5; ++i) arr[i] = i + 1;
    return arr;
}

// 使用后必须释放
int main() {
    int* arr = get_dynamic_ptr();
    // ... 使用
    delete[] arr; // 必须手动释放，否则内存泄漏
    return 0;
}
```

⚠ 风险：

- 静态数组是全局共享的，多线程场景下可能有数据竞争
- 动态分配的数组必须手动 `delete[]`，极易造成内存泄漏

4. 进阶方案：用结构体/类包装数组

通过自定义结构体封装数组，可实现值语义，并附加额外信息（如数组大小）。

Code block

```
struct IntArray {
    int data[5];
    int size = 5; // 附加数组大小信息
};

IntArray get_struct_array() {
    IntArray arr;
    for (int i = 0; i < 5; ++i) {
        arr.data[i] = i + 1;
    }
    return arr;
}
```

✅ 优点：

- 可封装额外业务逻辑
- 支持值拷贝，无生命周期问题

⚠ 限制：不如 `std::array` 简洁，需手动定义结构体

💡 核心总结与选型建议

方案	适用场景	优点	缺点
`std::array<T,N>`	编译期确定大小的固定数组	高效、安全、现代C++风格	大小不可变
`std::vector<T>`	运行期可变大小的动态数组	灵活、自动内存管理	轻微性能开销
指针（静态/动态）	兼容旧代码、极致性能场景	语法简单、性能最高	生命周期风险、内存泄漏
结构体包装	需要封装额外信息的场景	可扩展性强	代码冗余

推荐优先级：

`std::vector` （灵活） > `std::array` （固定大小） > 结构体包装 > 指针（尽量避免）

你要我帮你写一个**完整的对比示例代码**，把这几种方式的用法和坑都演示出来吗？这样你可以直接编译运行，直观感受差异。

（注：文档部分内容可能由 AI 生成）